King Mongkut's University of Technology Thonburi
Faculty of Engineering, Department of Computer Engineering
CPE 333 Operating Systems, 1/2026

PROBLEM SESSION 2: Process Creation & Pipes.

This is the problem session for the group of up to **3-4** students.
Developing the labs in unix/linux environment is easier. So you might want to either install dual boot for windows/linux or create a linux VM using VMware or VirtualBox. For those who use Mac OS, your environment is already good for this lab. Note that some system calls, functions, and command line involved in this lab might not work on windows.

1. Write two programs as fork() example from process lecture slide:

    1.1. The 'helloworld' one
    1.2. the one that modified with wait() function

run the program, observe and discuss the difference in the output between the two programs.

2. With help of sleep() function, you can make sure parents die before child do, and of course you can make sure parents die after child. Use 'ps' command to examine the process status (or you can also use 'ps -ef' to see parent of each process). There should be differences in the outputs. Examine the process list when:

    2.1. child is dead while parent is still running
    2.2. Parents are dead while child is running.

Discuss the differences between 2 scenarios. Report the child's parent process ID in both scenarios. Note that if you execute this lab correctly, you will notice that in some situations, certain processes are tagged as <defunct>. Explain the meaning, consequences, and causes of this phenomenon.

3. Find out how many processes you can fork. You are going to:

    3.1. write a program that forks until it cannot fork anymore. Then the program will report how many times the fork() is successfully called.
    3.2. Run some programs that would stay running in the system. See the number of processes you can fork again. After that, stop all the programs that you created after running the first fork count and see how many fork() can be successfully called again. Discuss the result

Hint
You can use a recursive function that parent wait for child and child forks a child, until child cannot fork anymore. Keep the child counting and report the last number when the fork is

not successfully called.
***caution *** this program can crash your system if you do not control your fork() well.**

4. Write a program that creates a pipe using the pipe() function and then spawns a new child process using the fork() function. The parent and child processes should communicate through the pipe. Note that the file descriptors of the pipe are shared between the parent and child. The sender should therefore close the read-end of the pipe and the receiver should close the write-end of the pipe before trying to communicate.

Let the child be the sender and the parent the receiver. The child should first print out "Child" and its PID, and then send a message through the pipe consisting of "Hello from child" and its PID. The parent should first print out "Parent" and its PID. After it reads the message, it should print out the received message.

Expected output:

Child: Child PID: xxxxxx
Parent: Parent PID: yyyyyy
Parent: Hello from child PID: xxxxxx

5. Test the situations where:

    5.1. sender tries to read back its own message
    5.2. receiver reads before sender sends
    5.3. sender sends several messages before receiver reads

Write down what you observed from the experiment and discuss the results.

Hint
These are the system calls you might need to study:

int pipe(int *fd[2]*);
pipe() takes a single argument, which is an array of two integers. If successful, the array will contain two new file descriptors to be used for the pipe, where *fd[0]* is set up for reading and *fd[1]* is set up for writing. It returns 0 on success, and -1 on error with errno set appropriately.

ssize_t read(int *fd*, void **buf*, size_t *count*);
read() reads up to *count* bytes from the file descriptor *fd* into the buffer *buf*. It returns the number of bytes read if successful, and -1 on error with errno set appropriately.

ssize_t write(int *fd*, const void **buf*, size_t *count*);
write() writes up to *count* bytes from the buffer *buf* to the file descriptor *fd*. It returns the number of bytes written if successful, and -1 on error with errno set appropriately.

int close(int *fd*);
close() closes the file descriptor *fd*. It returns 0 on success, and -1 on error with errno set

appropriately.

Submission:

A PDF report contains:

- The copy of C source code, with overview explanation of how it works
- Results
- Discussions

Submit your PDF report to LEB2 before the deadline. (No need to submit the C file)